

Recognising Handwritten Digits with a Convolutional Neural Network

AI Foundations Bootcamp — Final Capstone Report

Student	■■■■■ ■■■■■■
Instructor	■■■■■■■■■ ■■■■ ■■■■■■
Course	AI Foundations Bootcamp — Week 10
Dataset	MNIST Digit Recognizer (Kaggle)

1. Introduction

For this capstone project I used the **MNIST dataset**, which contains 70,000 grayscale images (28×28 pixels) of handwritten digits from 0 to 9. The dataset is perfectly balanced, with approximately 7,000 samples per class.

This project aimed to answer three questions:

- 1. Can a simple CNN read handwritten digits with high accuracy?
- 2. Which digits does it confuse most often?
- 3. How does Dropout regularisation affect generalisation?

2. Planning Summary

Problem Statement

"We want to automatically recognise handwritten digits for form-processing teams so that data-entry staff can focus on exceptions, by building a model that classifies a 28×28 grayscale image into one of ten digit classes (0–9)."

Success Metrics

Type	Metric	Target
Technical	Test accuracy	≥ 99%
Technical	Inference latency (CPU)	< 50 ms
Business	Forms auto-processed without human	≥ 95%

Risk Register

#	Risk	Mitigation
1	Overfitting	Dropout(0.25); monitor val loss each epoch
2	Confusable digits (4↔9, 3↔5)	Confusion matrix + error analysis
3	Domain gap (clean vs real handwriting)	Noted as limitation; add augmentation
4	Reproducibility	Fixed random seeds (42) before every run

3. Development Summary

Data Split

Split	Samples	Purpose
-------	---------	---------

Training	54,000	Model learning
Validation	6,000	Hyperparameter tuning (val_accuracy)
Test	10,000	Final evaluation — touched ONCE only

Model Architecture (Final)

Input → (28, 28, 1)
Conv2D(64 filters, 3×3, ReLU) → MaxPooling2D
Conv2D(128 filters, 3×3, ReLU) → MaxPooling2D
Flatten → Dropout(0.25)
Dense(64, ReLU) → Dense(10, Softmax)

Experiment Table

Each experiment changed exactly ONE variable to isolate its effect:

Experiment	Epochs	Dropout	Val Acc
Baseline (32/64 filters)	5	0.0	98.53%
More epochs (32/64)	10	0.0	98.70%
Dropout 0.25 (32/64)	10	0.25	98.90%
Dropout 0.25 + larger filters (64/128)	10	0.25	99.10% ✓

4. Key Findings

Finding 1 — High accuracy achieved

The final model reached **99.1% accuracy** on the untouched test set, meeting the $\geq 99\%$ technical target. Because MNIST is perfectly balanced across all 10 classes, this accuracy figure is trustworthy and not inflated by class imbalance.

Finding 2 — Errors cluster around visually similar digits

The confusion matrix showed the most errors between digit pairs **4↔9** and **3↔5**. Inspecting the misclassified images confirmed these were unusually written or ambiguous digits — cases that even a human might hesitate on.

Finding 3 — Dropout improved generalisation

Without Dropout, the training accuracy climbed to ~99.5% while validation accuracy plateaued at ~98.7% — a classic sign of overfitting. Adding Dropout(0.25) narrowed this gap and pushed validation accuracy above 99%.

Finding 4 — Larger filters extract richer features

Doubling the filter counts from (32, 64) to (64, 128) gave an additional +0.2% improvement in validation accuracy, at the cost of ~2× more parameters and slightly longer training time per epoch.

See reports/training_curves.png for accuracy/loss charts and reports/confusion_matrix.png for the full 10×10 confusion matrix.

5. Limitations

- **Clean data only:** The model was trained and tested exclusively on MNIST's clean, centred, well-sized digits. Performance on messy or rotated real-world handwriting is expected to be lower.
- **No data augmentation:** Adding random rotations ($\pm 10^\circ$) and pixel shifts (± 2 px) would help the model generalise to more varied input styles.
- **Fixed image size:** The pipeline requires a 28×28 input. Any real-world system would need a preprocessing step to detect and crop individual digits from a document.

6. What I Would Do Next

- **Add data augmentation** (rotations and shifts) and compare accuracy with and without it.
- **Test on real handwriting** — photograph my own handwritten digits and run them through the model.
- **Try a deeper architecture** such as ResNet-20 to see whether accuracy improves further.
- **Build a simple web interface** using Flask or Streamlit for real-time digit prediction.

7. Reflection

What worked well

The CNN architecture came together quickly thanks to the clear project structure. The experiment table was especially useful — by changing only one variable at a time I could clearly see the contribution of each improvement.

What did not work initially

The first training run without Dropout overfit noticeably. Training accuracy climbed to 99.5%+ while validation accuracy plateaued around 98.7% — the classic sign.

What I would change

I would add early stopping from the very beginning to avoid unnecessary epochs, and start with data augmentation rather than treating it as an afterthought.

Biggest learning

A single accuracy number is not enough. The confusion matrix and error analysis told a much richer story about where and why the model fails.

Student	Instructor
■■■■■ ■■■■■■	■■■■■■■■■ ■■■■ ■■■■■■
AI Foundations Bootcamp	Week 10 Capstone